

Code Explanation

Testing Batch Processing with PySpark

This test suite is designed to validate the functionality of the ``clean_data`` function from the ``src.batch_processing`` module. The ``clean_data`` function is expected to clean customer and order data by removing duplicates and nulls, and calculating the total amount for each order.

The test suite utilizes PySpark to create sample customer and order data, applies the ``clean_data`` function, and then asserts that the resulting dataframes meet the expected conditions.

Overview of the Code

The provided code is a PySpark test suite written using the Pytest framework. It tests the ``clean_data`` function, which is responsible for cleaning customer and order data. The test suite creates sample data, applies the ``clean_data`` function, and verifies the results.

Step 1: Creating a Spark Session for Testing

This step creates a Spark session with the necessary configurations for testing. The Spark session is created using the ``SparkSession.builder`` and is configured to use the Delta Lake catalog.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
    return SparkSession.builder
        .appName("Test Customer Order Processing")
        .master("local[*]")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Defining the Test Function for clean_data

This step defines the test function ``test_clean_data``, which tests the ``clean_data`` function. It creates sample customer and order data, applies the ``clean_data`` function, and verifies the results.

```
def test_clean_data(spark):
    """Test the clean data function."""
```

Step 3: Creating Sample Customer and Order Data

This step creates sample customer and order data with nulls and duplicates. The data is used to test the ``clean_data`` function.

```
# Create sample customer data with nulls and duplicates
customer_data = [
    (1, "John Doe", "john@example.com", "North"),
    (1, "John Doe", "john@example.com", "North"), # Duplicate
    (2, None, "jane@example.com", "South"), # Null name
    (3, "Bob Smith", "bob@example.com", "East")
```

```

]
customer_schema = ["CustId", "Name", "EmailId", "Region"]
customer_df = spark.createDataFrame(customer_data, customer_schema)

# Create sample order data with nulls and duplicates
order_data = [
    (101, "Item1", 10.0, 2, "2023-01-01", 1),
    (101, "Item1", 10.0, 2, "2023-01-01", 1), # Duplicate
    (102, "Item2", None, 3, "2023-01-02", 2), # Null price
    (103, "Item3", 15.0, 1, "2023-01-03", 3)
]
order_schema = ["OrderId", "ItemName", "PricePerUnit", "Qty", "Date",
                "CustId"]
order_df = spark.createDataFrame(order_data, order_schema)

```

Step 4: Applying the clean_data Function

This step applies the `clean\_data` function to the sample customer and order data.

```

# Clean the data
clean_customer_df, clean_order_df = clean_data(customer_df, order_df)

```

Step 5: Verifying the Results

This step verifies that the resulting dataframes meet the expected conditions. It checks the count of rows in the cleaned dataframes and the calculated total amount for each order.

```

# Verify customer data cleaning
assert clean_customer_df.count() == 2 # Should remove duplicates and nulls

# Verify order data cleaning
assert clean_order_df.count() == 2 # Should remove duplicates and nulls

# Verify TotalAmount calculation
total_amount = clean_order_df.filter(F.col("OrderId") == 101).select(
    ("TotalAmount").collect()[0][0]
)
assert total_amount == 20.0 # 10.0 * 2 = 20.0

# Verify TotalAmount calculation for another record
total_amount = clean_order_df.filter(F.col("OrderId") == 103).select(
    ("TotalAmount").collect()[0][0]
)
assert total_amount == 15.0 # 15.0 * 1 = 15.0

```